

Báo cáo

Bài 4. Phân tích mã độ (tĩnh + động)

Phần Lý thuyết

MỤC LỤC

1. Phân tích động.....	3
1.1 Định nghĩa.....	3
1.2 Mục đích.....	3
1.3 Ưu điểm.....	3
1.4 Hạn chế.....	3
1.5 So sánh với Static analysis.....	3
2. Các công cụ phân tích động.....	4
2.1 Process Hacker.....	4
2.2 Process Explorer.....	4
2.3 Process Monitor.....	4
2.4 Autoruns.....	4
2.5 Wireshark.....	5
2.6 HTTP Debugger.....	5
3. Các nhóm hành vi quan trọng khi phân tích động.....	7
3.1 Process Injection.....	7
3.2 File system.....	7
3.3 Registry.....	7
3.4 Persistence.....	7
3.5 Network.....	7
3.6 Anti-analysis / Evasion.....	8
3.7 DLL Loading.....	8

1. Phân tích động

Nguồn tham khảo: [CrowdStrike](#).

1.1 Định nghĩa

Phân tích động (Dynamic analysis) là việc thực thi mã độc trong môi trường an toàn (sandbox).

1.2 Mục đích

- Theo dõi trực tiếp cách thức hoạt động của mã độc.
- Cô lập mã độc trong một hệ thống đóng để triệt tiêu nguy cơ lây nhiễm sang máy tính đang phân tích hoặc phát tán ra mạng nội bộ.

1.3 Ưu điểm

- Quan sát rõ hành vi, truy ra bản chất của mã độc.
- Định vị được đoạn payload, vùng chứa mã thực thi cụ thể.

1.4 Hạn chế

- Cần triển khai môi trường sandbox cẩn trọng, đầy đủ tài nguyên để chạy mã độc.
- Kẻ tấn công có các kỹ thuật để nhận biết nếu phần mềm của chúng đang chạy trong sandbox, ví dụ như dùng các API QueryPerformanceFrequency, QueryPerformanceCounter để theo dõi thời gian thực thi. Nếu phát hiện ra sandbox, mã độc sẽ tự hủy hoặc giả vờ là file sạch để đánh lừa hệ thống.
- Mã độc có thể sử dụng cơ chế hẹn giờ/ngủ đông hoặc chờ các hành vi cụ thể từ người dùng thật (di chuyển chuột, nhấn phím...) mới kích hoạt.

1.5 So sánh với Static analysis

Static analysis phân tích mã độc dựa trên các thông tin khai thác được từ chương trình mà hoàn toàn không thực thi nó. Dynamic analysis thực thi chương trình mã độc để khai thác thêm thông tin.

2. Các công cụ phân tích động

2.1 [Process Hacker](#)

Mục đích chính: Giám sát tài nguyên hệ thống, gỡ lỗi phần mềm, phát hiện mã độc.

Loại dữ liệu thu thập được:

- Lượng tài nguyên hệ thống mà 1 tiến trình cụ thể tiêu thụ.
- Tìm và xóa định danh (handle) của tiến trình.
- Quản lý kết nối mạng của tiến trình kể cả khi tiến trình đã dừng.
- Theo dõi thông tin quyền truy cập, tiêu thụ tài nguyên ổ cứng; thông tin tiêu thụ tài nguyên mạng, GPU.
- Truy vết stack của 1 tiến trình với sự hỗ trợ của kernel-mode, WOW64, .NET.

Ví dụ thông tin có giá trị trong phân tích malware: 1 tiến trình tốn lượng tài nguyên lớn hơn hẳn các tiến trình khác; 1 tiến trình mở kết nối mạng đến 1 domain đáng ngờ...

2.2 [Process Explorer](#)

Mục đích chính: Giám sát chuyên sâu tiến trình để xác định các handle mà chúng đã mở và DLL đã nạp.

Loại dữ liệu thu thập được:

- Cây tiến trình cha/con, loại tiến trình (service, tiến trình .NET...).
- Các DLL đã tải, tệp được ánh xạ vào bộ nhớ, và các handle đang mở trở tới các đối tượng kernel (tệp tin, thư mục...).
- Chỉ số tiêu thụ CPU chi tiết (được tính toán chính xác dựa trên số chu kỳ CPU hoặc context switch), cùng số liệu và biểu đồ trực quan về mức sử dụng bộ nhớ và hoạt động I/O.
- Bối cảnh bảo mật của tiến trình và các endpoint kết nối mạng.
- Danh sách luồng, địa chỉ bắt đầu và stack của tiến trình; tạo process dump.

Ví dụ thông tin có giá trị trong phân tích malware: winword.exe sinh tiến trình con cmd.exe; svchost.exe gọi đến lpk.dll từ %APPDATA% thay vì C:\Windows\System32\...

2.3 [Process Monitor](#)

Mục đích chính: Log chi tiết mọi hoạt động cấp thấp của hệ thống (tệp tin, registry, tiến trình và mạng) nhằm chẩn đoán và tìm ra nguyên nhân gốc rễ của các sự cố máy tính.

Loại dữ liệu thu thập được:

- Ghi lại toàn bộ hoạt động của hệ thống tệp và các thao tác trên Registry.
- Thông tin về tiến trình: đường dẫn tệp, dòng lệnh, cây tiến trình, tên người dùng, session ID, thread stack...
- Chi tiết các dữ liệu đầu vào và đầu ra cho từng thao tác riêng biệt.
- Log toàn bộ các thao tác diễn ra trong suốt quá trình khởi động máy tính.

Ví dụ thông tin có giá trị trong phân tích malware: excel.exe drop 1 file .dll vào %APPDATA%; 1 file tự ghi path của nó vào key Run; 1 file liên tục truy vấn HKCU\Software\Classes\CLSID\...

2.4 [Autoruns](#)

Mục đích chính: Quản lý các chương trình được cấu hình để tự động chạy khi hệ thống khởi động, khi người dùng đăng nhập, hoặc khi khởi chạy các ứng dụng khác.

Loại dữ liệu thu thập được:

- Các chương trình thư mục Startup và các khóa Registry (như Run, RunOnce).
- Các mục đăng nhập hệ thống (logon entries).
- Các thành phần mở rộng của Windows Explorer (Explorer shell extensions/add-ons).
- Tiện ích bổ sung của trình duyệt (Internet Explorer add-ons), thanh công cụ, và Browser Helper Objects.
- Các dịch vụ hệ thống (Windows Services, auto-start services) và Winsock Layered Service Providers.
- Thông báo của Winlogon (Winlogon notifications/DLLs).
- Các tệp Appinit DLLs, mục thực thi lúc khởi động (boot execute images), image hijacks, và bộ giải mã đa phương tiện.
- Trạng thái chữ ký điện tử và mã băm của tệp tin để kiểm tra qua VirusTotal.

Ví dụ thông tin có giá trị trong phân tích malware: (Services) 1 dịch vụ hệ thống như Windows System Update có path trong %TEMP% thay vì C:\Windows\System32\; 1 file .dll trong AppData\Roaming có entry trong HKCU\Software\Microsoft\Windows\CurrentVersion\Run...

2.5 Wireshark

Mục đích chính: Điều tra các vấn đề về bảo mật mạng; Xác minh hoạt động của các ứng dụng mạng và gỡ lỗi quá trình triển khai giao thức; Nghiên cứu và học tập chuyên sâu về cấu trúc bên trong của các giao thức mạng.

Loại dữ liệu thu thập được:

- Bắt các gói tin trực tiếp từ một giao diện mạng.
- Mở/import dữ liệu gói tin từ các tệp lưu trữ (tcpdump/Wireshark) hoặc tệp văn bản chứa hex dump của dữ liệu gói tin.
- Bóc tách và hiển thị thông tin giao thức của từng gói tin.
- Sàng lọc, tìm kiếm và đánh dấu các gói tin.
- Lưu, xuất dữ liệu gói tin và tạo ra các số liệu thống kê mạng đa dạng.

Ví dụ thông tin có giá trị trong phân tích malware: nhiều gói tin gửi truy vấn DNS vô nghĩa, gần như xqkaght.com (dấu hiệu DNS Exfiltration hoặc Domain Generation Algorithm; payload của nhiều các gói tin ICMP chứa base64...

2.6 HTTP Debugger

Mục đích chính: Là một công cụ bắt gói tin HTTP không yêu cầu thiết lập proxy, giúp thu thập, giám sát và phân tích chuyên sâu các thành phần dữ liệu mạng (HTTP header, cookie, tham số POST, nội dung HTTP và CORS header) từ bất kỳ trình duyệt hoặc ứng dụng máy tính nào.

Loại dữ liệu thu thập được:

- Dữ liệu từ HTTP/1.1, HTTP/2, WebSocket, Server-Sent Events và gRPC.
- Chi tiết Yêu cầu/Phản hồi: HTTP headers, cookies, URL, mã hex, thời gian thực thi (request timing).
- Streaming protocols: frame, thông điệp, metadata (hướng gửi, loại, kích thước, độ nén, payload đã được giải mã).
- Số liệu tổng hợp theo phiên làm việc (session): lưu lượng theo tên miền, số lượng yêu cầu, loại nội dung, kích thước và thời gian phản hồi.

Ví dụ thông tin có giá trị trong phân tích malware: trường cookie của gói tin chứa 1 chuỗi base64 dài; quan sát Session thấy rundll32.exe liên tục tải 1 file jquery đều đặn 45s/lần...

3. Các nhóm hành vi quan trọng khi phân tích động

3.1 [Process Injection](#)

Là kỹ thuật chèn mã độc hại (shellcode, DLL) vào không gian bộ nhớ của một tiến trình hợp pháp đang chạy trên hệ thống.

- Ví dụ: Mã độc cấp phát bộ nhớ trống trong tiến trình explorer.exe (sử dụng API VirtualAllocEx), ghi payload vào đó (WriteProcessMemory), và ra lệnh cho explorer.exe thực thi mã này (CreateRemoteThread).
- Tầm quan trọng: Giúp mã độc ẩn mình dưới vỏ bọc của các tiến trình hệ thống đáng tin cậy. Điều này khiến AV/EDR khó phát hiện.

3.2 [File system](#)

Là các thao tác tương tác với hệ thống tệp tin của hệ điều hành, bao gồm việc tạo mới, đọc, ghi, xóa hoặc sửa đổi thuộc tính của file và thư mục.

- Ví dụ: Mã độc rà quét toàn bộ thư mục C:\Users\ để tìm các file .docx (đọc file), hoặc tải xuống và lưu một file thực thi mới vào C:\Users\Public\payload.exe (ghi file/dropping).
- Tầm quan trọng: Việc theo dõi hành vi tệp tin giúp xác định mục đích cốt lõi của mã độc (ví dụ: ransomware sẽ mã hóa file, spyware sẽ thu thập tài liệu, dropper sẽ thả thêm mã độc).

3.3 [Registry](#)

Là các hành vi truy vấn, tạo mới, hoặc sửa đổi các khóa/giá trị trong cơ sở dữ liệu Windows Registry.

- Ví dụ: Mã độc truy vấn khóa HKLM\System\CurrentControlSet\Control\ComputerName để lấy định danh máy, hoặc sửa đổi HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings để vô hiệu hóa proxy của trình duyệt.
- Tầm quan trọng: phát hiện cách mã độc thay đổi cấu hình bảo mật, đánh cắp thông tin môi trường, thiết lập persistence, hoặc làm suy yếu các tính năng phòng thủ của hệ điều hành...

3.4 [Persistence](#)

Là các kỹ thuật thiết lập cơ chế tự động kích hoạt nhằm duy trì sự tồn tại của mã độc trên hệ thống kể cả khi người dùng khởi động lại máy tính.

- Ví dụ: Ghi đường dẫn của file mã độc vào khóa Registry Run, tạo một Windows Service mới chạy ngầm, hoặc thiết lập một Scheduled Task tự động kích hoạt vào một giờ cố định.
- Tầm quan trọng: Đây là cơ chế gần như luôn có của hầu hết mã độc để bám rễ lâu dài (đặc biệt là các chiến dịch APT). Nhận diện được cơ chế Persistence giúp xây dựng các kịch bản gỡ bỏ hoàn toàn mã độc ra khỏi hệ thống.

3.5 [Network](#)

Là các hoạt động giao tiếp truyền tải dữ liệu qua môi trường mạng (TCP/UDP, HTTP, DNS...) tới các địa chỉ IP hoặc tên miền bên ngoài.

- Ví dụ: Tiến trình sinh ra các truy vấn DNS liên tục đến các tên miền ngẫu nhiên, hoặc gửi yêu cầu HTTP POST chứa dữ liệu mã hóa đến máy chủ 185.x.x.x.

- Tầm quan trọng: Xác định được địa chỉ máy chủ điều khiển (C2 server), phát hiện luồng dữ liệu bị đánh cắp (Exfiltration), và cung cấp trực tiếp các IoC mạng để cấu hình luật chặn lọc trên Firewall/IDS.

3.6 Anti-analysis / Evasion

Là các thủ đoạn mã độc sử dụng để phát hiện môi trường phân tích (Sandbox, Virtual Machine, Debugger) nhằm tạm dừng hoạt động hoặc ẩn giấu hành vi thật.

- Ví dụ: Dùng cmd.exe tìm kiếm thư mục VMware Tools, truy vấn khóa SafeBoot\Option để xem máy có đang ở Safe Mode không, hoặc sử dụng hàm Sleep() lặp lại nhiều lần để kéo dài thời gian thực thi làm cạn kiệt tài nguyên của sandbox.
- Tầm quan trọng: Quan trọng trong phân tích động. Nếu không vô hiệu hóa được các lớp phòng vệ này, mã độc sẽ không bộc lộ hành vi độc hại, dẫn đến việc phân tích đánh giá sai.

3.7 DLL Loading

Là quá trình một tiến trình yêu cầu hệ điều hành nạp (load) các thư viện liên kết động (.dll) vào không gian bộ nhớ để sử dụng các hàm API bên trong đó.

- Ví dụ: Thay vì nạp thư viện hệ thống, tiến trình hợp pháp wss.exe lại bị đánh lừa để nạp một file libcef.dll giả mạo nằm cùng thư mục (DLL Side-Loading).
- Tầm quan trọng: Các DLL được nạp cho biết mục đích/hành vi tiếp theo của mã độc (VD: nạp winhttp.dll chứng tỏ chuẩn bị gọi mạng)..